Module 17 – Generative AI Fundamentals

1. Introduction to Generative AI

What is Generative AI?

Generative Artificial Intelligence (Generative AI or GenAI) is a branch of Artificial Intelligence that creates new content by learning patterns from existing data.

Unlike traditional AI systems that only classify or predict, Generative AI can produce original outputs such as:

- Text
- Source code
- Images
- Audio
- Videos
- Documents

Generative AI uses advanced deep learning models, particularly **Large Language Models (LLMs)** and **Generative Models**, to understand user input and generate human-like responses.

How Does Generative AI Work?

Generative AI is trained on massive datasets consisting of books, articles, websites, code repositories, and other publicly available information.

During training, the model learns:

- Language patterns
- Grammar
- Programming syntax
- Relationships between words and concepts
- Contextual understanding

When a user provides a prompt, the model predicts the most appropriate sequence of words or code based on what it has learned.

Generative AI vs Traditional (Discriminative) AI

Traditional AI mainly focuses on recognizing or classifying existing data, whereas Generative AI creates new content.

Traditional (Discriminative) AI	Generative AI
Predicts or classifies data	Creates new content
Answers "What is this?"	Answers "Create something new."

Used for spam detection, fraud detection, image classification	Used for chatbots, code generation, image creation, document generation
Output is a prediction or label	Output is newly generated content

Applications of Generative AI

Generative AI has applications across many domains.

Text Generation

Used for:

- Writing articles
- Summarizing documents
- Drafting emails
- Content creation
- Translation

Examples:

- ChatGPT
- Google Gemini
- Claude

Code Generation

Generates:

- Functions
- Algorithms
- Boilerplate code
- Documentation
- Test cases

Examples:

- GitHub Copilot
- Amazon Q Developer
- Codeium

Image Generation

Creates images from text prompts.

Examples:

- DALL·E
- Midjourney
- Stable Diffusion

Chatbots and Virtual Assistants

Provide conversational assistance for:

- Customer support
- Education
- Healthcare
- Programming help

Examples:

- ChatGPT
- Microsoft Copilot
- Google Gemini

History and Evolution of Generative AI

Generative AI has evolved significantly over the past several decades.

Year	Milestone
1960s	Early chatbots such as ELIZA demonstrated rule-based conversations.
2014	Generative Adversarial Networks (GANs) introduced realistic image generation.
2017	Transformer architecture revolutionized Natural Language Processing (NLP).
2020	GPT-3 demonstrated powerful large-scale language generation capabilities.
2022	ChatGPT popularized conversational AI for the general public.
2022–Present	GitHub Copilot, Google Gemini, Claude, DALL·E, and other advanced GenAI tools expanded AI-assisted development and content creation.

Benefits of Generative AI

- Increases productivity.
- Automates repetitive tasks.
- Assists software development.
- Enhances creativity.
- Accelerates content generation.
- Supports learning and research.
- Improves customer interaction.

Limitations of Generative AI

- May generate incorrect information (hallucinations).
- Can reflect biases present in training data.
- Lacks human judgment and common sense.
- Requires verification of generated content.
- May produce insecure or inefficient code if not reviewed.

2. Prompt Engineering

What is Prompt Engineering?

Prompt Engineering is the process of designing clear and effective instructions (prompts) to guide an AI model toward generating accurate, relevant, and useful outputs.

The quality of the AI's response depends heavily on the quality of the prompt.

Why is Prompt Engineering Important?

Effective prompts help:

- Improve response accuracy.
- Reduce ambiguity.
- Generate structured outputs.
- Save time.
- Produce better code and documentation.

Prompting Techniques

1. Zero-Shot Prompting

The model is given only the task without any examples.

Example

Write a C# method to reverse a string.

The model generates the solution directly based on its existing knowledge.

2. Few-Shot Prompting

The prompt includes one or more examples before asking the model to perform a similar task.

Example

Input:
5 + 3

Output:
8

Input:
10 + 6

Output:
16

Now solve:

15 + 20

The examples help the model understand the expected format and improve response accuracy.

3. Chain-of-Thought Prompting

The model is encouraged to reason through a problem step by step before producing the final answer.

Example

Solve the following problem step by step before giving the final answer.

This approach is particularly useful for complex reasoning tasks such as mathematics, debugging, and algorithm design.

Prompt Engineering Best Practices

To obtain better responses:

- Write clear and specific instructions.
- Provide sufficient context.
- Specify the desired output format.
- Mention constraints or requirements.
- Break large tasks into smaller prompts.
- Refine prompts based on previous responses.
- Verify AI-generated outputs before use.

Example of a Poor Prompt

Write code.

This lacks context and is likely to produce an unclear or generic response.

Example of a Better Prompt

Write a C# method that accepts an integer array and returns the largest element. Include comments and explain the time complexity.

This prompt is specific, includes the programming language, task, and expected output details.

Ethical Considerations in Prompt Engineering

When using Generative AI responsibly:

- Avoid generating biased or discriminatory content.
- Protect confidential or personal information.

- Verify factual accuracy.
- Respect copyright and intellectual property.
- Use AI as an assistant rather than replacing human judgment.

Hands-on Example

Prompt

Write a C# method that checks whether a given number is prime. Include comments and explain the algorithm.

A well-structured prompt helps the AI generate readable, documented, and efficient code.

3. Introduction to GitHub Copilot

What is GitHub Copilot?

GitHub Copilot is an AI-powered coding assistant that helps developers write code faster by providing intelligent code suggestions directly within the code editor.

It acts as an **AI pair programmer**, assisting with coding, documentation, testing, and debugging.

How GitHub Copilot Works

GitHub Copilot analyzes:

- The current file.
- Existing code.
- Comments.
- Function names.
- Project context.

It then predicts and suggests the most relevant code completions using AI models developed by OpenAI.

Features of GitHub Copilot

- Code completion.
- Function generation.
- Documentation generation.
- Test case generation.
- Code explanation.
- Refactoring suggestions.
- Error correction.
- Boilerplate code generation.

Supported IDEs

GitHub Copilot integrates with several popular development environments, including:

- Visual Studio Code
- Visual Studio
- JetBrains IDEs (IntelliJ IDEA, PyCharm, WebStorm, etc.)
- Neovim

Supported Programming Languages

GitHub Copilot supports many languages, including:

- Python
- Java
- JavaScript
- TypeScript
- C#
- C++
- Go
- PHP
- Ruby
- Kotlin
- SQL

4. Setup and Configuration

Installing GitHub Copilot in Visual Studio Code

Typical setup process:

1. Install **Visual Studio Code**.
2. Open the **Extensions** view.
3. Search for **GitHub Copilot**.
4. Install the extension.
5. Sign in with your GitHub account.
6. Authorize GitHub Copilot.
7. Open a programming project and start coding.

Copilot will begin suggesting code automatically as you type.

First Coding Task

Example:

Write the following comment:

// Create a method to calculate factorial

GitHub Copilot will automatically suggest the complete implementation.

Press **Tab** to accept the suggestion.

5. Core Features and Capabilities

Code Suggestions and Completions

As you type, GitHub Copilot predicts the next lines of code.

Example:

def factorial(n):

Copilot may automatically generate the entire function body.

Writing Functions from Comments

Developers can describe functionality using comments.

Example:

// Read all student records from the database

Copilot generates the corresponding code implementation.

Boilerplate Code Generation

Copilot can automatically generate repetitive code such as:

- Classes
- Constructors
- API controllers
- CRUD operations
- Configuration files

This reduces development time.

Automatic Documentation

Copilot can generate:

- Function comments.
- XML documentation.
- Javadoc.
- Parameter descriptions.

Example:

```
/**
 * Calculates the average marks of students.
 */
```

Refactoring Existing Code

Copilot can suggest improvements such as:

- Simplifying loops.
- Improving readability.
- Reducing duplicate code.
- Optimizing algorithms.

Test Case Generation

GitHub Copilot can generate unit tests for existing functions.

Example:

Given a calculator function, Copilot can automatically suggest:

- Positive test cases.
- Negative test cases.
- Edge cases.
- Boundary value tests.

6. Security and Ethical Considerations

Risks of AI-Generated Code

Although AI-generated code is useful, developers must review all generated code carefully.

Possible risks include:

- Security vulnerabilities.
- Incorrect logic.
- Inefficient algorithms.
- Outdated programming practices.
- Hallucinated APIs or libraries.

Licensing and Attribution

AI-generated code may resemble publicly available code used during model training.

Developers should:

- Review generated code.
- Respect software licenses.
- Avoid blindly copying large code blocks.
- Ensure compliance with organizational policies.

Data Privacy

When using GitHub Copilot:

- Code context may be processed to generate suggestions.
- Avoid entering confidential information, passwords, API keys, or sensitive business logic into prompts.
- Follow your organization's security and privacy guidelines.

Responsible Use of GitHub Copilot

Best practices include:

- Review every AI-generated suggestion.
- Test generated code thoroughly.
- Validate security and performance.
- Understand the generated code before using it.
- Use AI as a productivity tool, not as a replacement for software engineering knowledge.

Best Practices for Using Generative AI

- Write clear, specific, and context-rich prompts.
- Use Zero-Shot, Few-Shot, or Chain-of-Thought prompting based on the complexity of the task.
- Verify AI-generated content for correctness, security, and efficiency.
- Avoid sharing confidential or sensitive information in prompts.
- Use GitHub Copilot to automate repetitive coding tasks while maintaining human oversight.
- Refactor and test AI-generated code before deploying it to production.
- Be mindful of ethical considerations such as bias, privacy, and licensing.
- Treat AI as a collaborative assistant that enhances developer productivity rather than replacing critical thinking.

Quick Revision Points

- **Generative AI** creates new content such as text, code, images, and audio by learning patterns from large datasets.
- Unlike **Traditional (Discriminative) AI**, which classifies or predicts data, Generative AI generates original outputs.
- Major applications of Generative AI include **text generation**, **code completion**, **image generation**, and **AI-powered chatbots**.
- Key milestones in the evolution of Generative AI include **ELIZA (1960s)**, **GANs (2014)**, **GPT-3 (2020)**, **ChatGPT (2022)**, and **GitHub Copilot**.
- **Prompt Engineering** is the practice of designing effective prompts to improve AI-generated responses.
- Common prompting techniques include **Zero-Shot Prompting**, **Few-Shot Prompting**, and **Chain-of-Thought Prompting**.
- Effective prompts should be **clear, specific, context-rich, and iterative**, while avoiding bias and protecting sensitive information.

- **GitHub Copilot** is an AI-powered coding assistant that provides code completions, function generation, documentation, refactoring suggestions, and test case generation within supported IDEs.
- GitHub Copilot integrates with environments such as **Visual Studio Code**, **Visual Studio**, and **JetBrains IDEs**, supporting languages including **Python**, **Java**, **JavaScript**, **C#**, and **SQL**.
- AI-generated code should always be reviewed, tested, and validated to address potential issues related to **security**, **privacy**, **hallucinations**, and **software licensing**.